

SARATHI

Personal Lifecycle AI Assistant

"The Charioteer of Your Life"

Product Requirements Document

Version 1.0 | April 2026 | CONFIDENTIAL

1. Product Overview

1.1 Vision

Sarathi is a personal lifecycle AI assistant that guides you from the moment your alarm rings in the morning to the moment you fall asleep at night. Named after the charioteer — as Krishna was Arjuna's Sarathi in the Mahabharata — the product acts as an intelligent, proactive guide that knows your goals, understands your patterns, tracks your progress, and holds you accountable.

Sarathi is not a chatbot. It is not a reminder app. It is an always-on, context-aware companion that learns who you are over time and actively steers your life toward the goals you set for yourself.

1.2 Problem Statement

Many people — especially students and young professionals — have the talent and the intention to succeed, but struggle with discipline, memory, and consistency. They forget tasks, get distracted, lose track of time, and lack a system that connects their daily actions to their long-term goals. Existing apps solve individual pieces (reminders, notes, habit trackers) but none act as a unified, intelligent, personalized life guide.

1.3 Target User (Phase 1)

The initial target user is the product creator himself — a college student with ambitious goals, irregular routines, and a tendency to forget or get distracted. Once the product demonstrably solves this personal problem, it will be opened to students broadly, then to working professionals.

1.4 Core Design Principles

- Proactive over Reactive — Sarathi reaches out to you; you don't always have to open the app
- Personalized over Generic — Every interaction is shaped by your history, goals, and patterns
- Accountability over Comfort — Sarathi tells you what you need to hear, not what you want to hear
- Privacy First — Sensitive on-device data is never sent to the cloud raw
- Lifecycle Aware — Sarathi understands your full day as a connected arc, not isolated events

2. Complete User Lifecycle & Feature Specification

2.1 Morning — Wake Up & Day Briefing

2.1.1 Alarm & Wake-Up Flow

When the user's alarm time arrives, Sarathi monitors whether the alarm has been dismissed. If the user misses or snoozes the alarm, Sarathi proactively opens on the phone screen with a motivational wake-up prompt tied directly to the user's own goals — not generic motivational content.

The tone is firm and accountable, not emotional or pampering. The user confirms they are awake by interacting with the app.

2.1.2 Morning Briefing

Once awake, Sarathi delivers a personalized morning briefing that includes:

- Today's goals and priorities based on what the user has previously set
- Gym / workout plan for the day including specific exercises if configured
- Breakfast suggestion based on the user's dietary goals
- Countdown timer: how much time before the user needs to leave for gym or college
- Real-time time-to-leave alerts so the user is never late
- A brief motivational context connecting today's plan to long-term goals

2.2 During the Day — Capture & Tracking

2.2.1 Quick Capture

While in college or elsewhere, the user can open the app at any time to quickly capture a thought, task, note, or event. Sarathi supports three input modes:

- Voice input — speak naturally, Sarathi transcribes and stores
- Text input — type quickly, Sarathi understands and categorizes
- Image input — photograph a whiteboard, assignment, or anything else; Sarathi processes and stores it with context

All captured items are stored in the user's personal memory and surfaced at the right time.

2.2.2 Location-Aware Triggers

Sarathi uses location awareness to detect when the user arrives home from college. Upon arrival, it proactively opens a check-in conversation.

2.3 Evening — Debrief & Work Session

2.3.1 Homecoming Conversation

When Sarathi detects the user is back home, it initiates a natural language conversation covering:

- How was the day? What happened?
- Summary of everything captured during the day
- Review of morning goals — what was achieved, what wasn't
- Transition into the evening work session

2.3.2 Evening Work Session Planning

After the debrief, Sarathi suggests a structured work plan for the evening based on remaining goals, pending tasks from captures, and the user's energy levels communicated in the conversation. It holds the user accountable to start working and stay focused.

2.4 Laptop — Focus Monitoring

2.4.1 Real-Time Screen Monitoring

When the user switches to their laptop for a work session, the Sarathi desktop web app activates screen monitoring. The local model analyzes what the user is doing on screen in real time.

- If the user is working on tasks aligned with their goals: no interruption
- If the user is wasting time (social media, games, unrelated browsing): Sarathi flags it with a firm, contextual nudge

Raw screen data is never stored or sent to the cloud. The local model extracts only behavioral patterns and summarized habits.

2.4.2 Productivity Pattern Learning

Over time, Sarathi learns the user's focus patterns:

- What music or environment helps them focus
- How long they can sustain deep work before a break
- What kinds of distractions they fall into most often
- Which times of day they are most productive

These patterns are stored as summarized habit data and used to personalize nudges, scheduling suggestions, and future planning.

2.5 Night — Sleep Debrief & Alarm Setup

2.5.1 Sleep Check-In Detection

Sarathi learns the user's typical sleep time based on pattern recognition. When it detects the user is likely heading to sleep (based on time patterns, inactivity, or a manual signal), it initiates the night debrief conversation.

2.5.2 Night Debrief Conversation

The evening conversation covers:

- What did you achieve today?
- What did you do wrong or could have done better?
- What does Sarathi observe as areas for improvement based on the full day's data?

- Encouragement and acknowledgment of wins — kept honest and not hollow

2.5.3 Alarm Setup

At the end of the night conversation, Sarathi asks what time the user wants to wake up, suggests an optimal time based on sleep goals, and sets the alarm on the user's phone automatically.

3. Technical Architecture

3.1 Architecture Overview

Sarathi uses a three-tier architecture: a local small model on the device, a RAG-powered personal memory database, and a shared cloud LLM as the primary brain. The local model acts as a privacy gatekeeper — it processes sensitive data on-device, extracts only safe summarized patterns, and decides what to send upstream.

Tier	Component	Responsibility
Tier 1 — Local	Small Gemma model on device	Privacy screening, real-time analysis, habit extraction
Tier 2 — Memory	RAG + Vector DB + PostgreSQL	Personal memory storage and retrieval
Tier 3 — Cloud	Large LLM (Claude API)	Intelligence, reasoning, personalized conversation

3.2 Local Model (Sarathi-Mini / Sarathi-Max)

Two model sizes are used depending on the device:

Sarathi-Mini (Mobile)	Sarathi-Max (Laptop)
Quantized Gemma 2B or smaller	Gemma 7B or larger
Runs via TensorFlow Lite / MediaPipe	Runs via Ollama or LM Studio
Battery-optimized, background-aware	Full hardware utilization
Handles screen monitoring lite, quick capture	Full screen monitoring, deep pattern analysis
User can toggle: always-on vs charging-only	Runs when laptop is active

Both models are periodically fine-tuned on the user's personal data using the supercomputer lab (NVIDIA DGX A100 system with 8x A100 80GB GPUs, 2TB system RAM, 15TB NVMe storage). Updated model checkpoints are pushed back to the device automatically — invisible to the user.

3.3 Memory System (RAG)

Every piece of information Sarathi learns about the user is stored in a structured personal memory system:

- Pinecone or Weaviate as the vector database for semantic retrieval
- PostgreSQL for structured data: goals, schedules, alarm preferences, task history
- Redis for caching active conversation context for fast lookups

When Sarathi needs to have a conversation with the user, it queries the memory system for relevant past context — recent captures, habit summaries, goal progress — and injects that context into the cloud LLM prompt. This is what makes Sarathi feel personalized without requiring a separate model instance per user.

3.4 Cloud LLM

The cloud model is Claude API (Anthropic) in Phase 1. It receives a structured prompt containing: the user's current context, relevant memory retrieved via RAG, the current moment in the lifecycle (morning/day/evening/night), and the conversation history. It generates natural, intelligent, personalized responses.

One shared cloud instance serves all users. Personalization is achieved through the memory context injected per user — not separate model instances.

3.5 Backend

Component	Technology
API Framework	Python — FastAPI
Primary Database	PostgreSQL
Cache	Redis
Vector Database	Pinecone or Weaviate
Object Storage	AWS S3 or MinIO (model checkpoints)
Authentication	JWT tokens with refresh flow
Encryption	AES-256 at rest, TLS 1.3 in transit
Deployment	Docker + Docker Compose (MVP)
Fine-tuning Pipeline	HuggingFace Transformers + PyTorch on DGX A100

3.6 Android App

Language	Kotlin
UI Framework	Jetpack Compose
Architecture	MVVM + Repository pattern
DI	Hilt
Local DB	Room (habit cache, offline support)
Networking	Retrofit + OkHttp
Local Model	TensorFlow Lite / MediaPipe
Voice Input	Android SpeechRecognizer API
Image Input	CameraX + ML Kit
Location	Google Fused Location Provider
Notifications	WorkManager + AlarmManager

3.7 Desktop (Web Interface)

Framework	React (TypeScript)
Styling	Tailwind CSS
State Management	Zustand or Redux Toolkit
API Communication	Axios + WebSockets for real-time
Screen Monitoring	Electron wrapper (later) or browser Screen Capture API
Local Model	Ollama or LM Studio (local API calls)

4. Phased Build Plan

Phase 1 — Core MVP (Personal Prototype)

Goal: Get the full lifecycle working for one user with Claude API and no local models. Validate that Sarathi actually solves the discipline and memory problem.

1. Set up FastAPI backend with PostgreSQL and Redis
2. Build user auth (JWT), goal setup, and basic profile
3. Implement RAG memory system with Pinecone
4. Build the morning briefing conversation flow (Claude API)
5. Build quick capture on Android — voice, text, image
6. Build location-based homecoming trigger
7. Build evening debrief and work session planning flow
8. Build night debrief and alarm-setting flow
9. Build basic React web interface for desktop conversations
10. Add basic screen monitoring (time-on-app tracking via Electron or browser API)

Phase 2 — Local Model Integration

Goal: Replace Claude API calls for real-time tasks with local Gemma models. Improve privacy and response speed.

11. Integrate Ollama/LM Studio on desktop — route screen monitoring to local model
12. Integrate TensorFlow Lite Gemma on Android
13. Build the privacy gateway: local model decides what to send to cloud
14. Test and tune the local-cloud handoff

Phase 3 — Self-Learning & Fine-Tuning

Goal: Make Sarathi genuinely personalized through periodic fine-tuning on user data.

15. Build data pipeline: collect anonymized habit summaries and conversation patterns
16. Set up fine-tuning infrastructure on DGX A100 supercomputer
17. Train personalized Gemma variants per user — push updated checkpoints to devices
18. Implement model versioning and rollback

Phase 4 — Scale & Product

Goal: Open Sarathi to other students, then professionals.

19. Multi-user backend hardening, rate limiting, billing
20. Onboarding flow for new users
21. Native desktop app (Electron wrapping the React frontend)
22. Analytics dashboard for users to review their own patterns
23. Expand fine-tuning pipeline for multiple concurrent users

5. Data & Privacy Architecture

5.1 What Stays Local

- Raw screen content — never leaves the device
- Live camera feed — processed on-device only
- Voice audio — transcribed locally before sending text
- Full conversation logs — only summaries and embeddings go to cloud

5.2 What Goes to Cloud

- Summarized habit patterns (e.g., "User tends to open YouTube during study sessions")
- Goal data and task captures
- Conversation text (after local model approves it for transmission)
- Anonymized behavioral embeddings for RAG retrieval

5.3 Encryption

- All data in transit: TLS 1.3
- All data at rest: AES-256
- Personal memory in vector DB: encrypted with per-user keys
- Model checkpoints: stored encrypted in object storage

6. Key Challenges & Mitigation

Challenge	Risk	Mitigation
Mobile battery drain from local model	High	User-configurable: always-on vs charging-only mode
Screen monitoring privacy	High	Local processing only; store habits not raw frames
Continuous fine-tuning complexity	High	Phase 3 only; use HuggingFace tooling on DGX A100
RAG context quality	Medium	Careful embedding design; test retrieval quality early
Conversation naturalness	Medium	Prompt engineering + Claude API; iterate on tone
Location trigger reliability	Medium	Geofencing with fallback manual trigger
Scope creep	High	Strict phase gating; ship Phase 1 before touching Phase 2

7. Success Metrics

Phase 1 Personal Prototype

- User (creator) uses Sarathi every day without having to force himself
- Morning briefing feels natural and motivating, not robotic
- At least 80% of daily captures are correctly surfaced in the evening debrief
- User reports measurable improvement in daily discipline and task completion
- Screen monitoring correctly flags distractions with less than 30 second delay

Phase 4 Product

- Day 30 retention > 60% for active users
- Users report Sarathi "knows them" within 2 weeks of use
- Average daily active usage > 5 meaningful interactions per day
- Net Promoter Score > 50

8. Out of Scope (Phase 1)

- College system integrations (AssignerVidya, Moodle) — deferred to later phase
- Social features or sharing
- Native desktop app (web interface in Phase 1; Electron wrapper in Phase 4)
- Custom on-device model fine-tuning by the user
- Multi-language support beyond Hindi and English
- Wearable integrations

— *End of Document* —

Sarathi PRD v1.0 | Confidential | April 2026